

EE445 Mod2-Lec1: Introduction to Linear Models

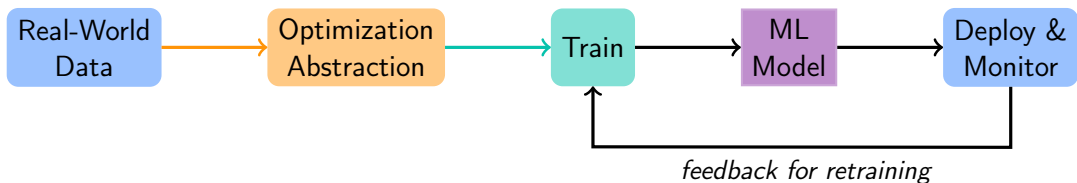
References:

- [VMLS]: Chapter 12

Agenda

- Introduction to linear models
- Least squares problem (linear regression)
- Example application: advertising purchases
- Optimality via vector calculus and linear algebra

ML workflow: Role of Linear Algebra & Optimization



This course so far:

- **Optimization & Linear Algebra:** Data $\rightarrow$ Optimization Abstraction via losses as norms
- **ML models:** Distance metric based models: k-NN, k-means

Coming up next:

- **Training:** Optimizing "convex" objectives (least squares)
- **ML models:** Linear models: linear regression, PCA, attention

ML workflow: Optimization Framework for Learning

Real world task involving data $\longrightarrow$ Optimization Problem $\longrightarrow$ ML model

Canonical Optimization Problem

$$\min_{\theta \in \Theta \subseteq \mathbb{R}^n} \ell(\theta)$$

Terminology:

- θ : Parameter/optimization or decision variable
- $\ell(\theta)$: Objective/Loss function; in ML will often depend on data Z ; write as $\ell(\theta; Z)$.
- $\Theta \subseteq \mathbb{R}^n$: Constraint set

Linear Models: Predicting Outputs from Inputs

- **Goal:** learn a function (aka "model") $f_{\theta}(x)$ that predicts quantity $y \in \mathbb{R}$ from input $x \in \mathbb{R}^d$.
 - For linear models:
-
- Given data (x_i, y_i) for $i = 1, \dots, n$, we choose θ to minimize prediction error.
 - Examples:
 - ▶ Predict house price from size, bedrooms, age.
 - ▶ Predict exam score from study hours.

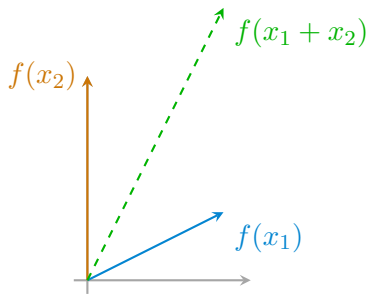
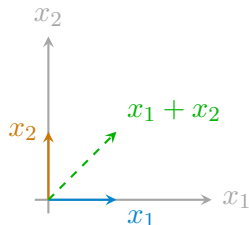
Linearity of Functions

Definition: A function $f : \mathbb{R}^d \rightarrow \mathbb{R}^m$ is **linear** if for all vectors x_1, x_2 and scalars α, β :

$$f(\alpha x_1 + \beta x_2) = \alpha f(x_1) + \beta f(x_2)$$

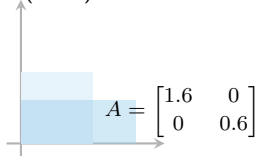
Intuition:

- Linear functions preserve addition and scaling.
- Geometrically, they stretch, rotate, or project vectors, but never bend or shift them.

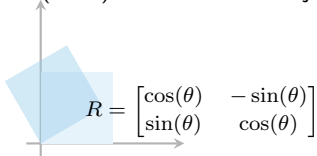


What Linear Models Can (and Cannot) Do

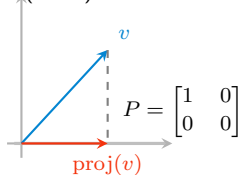
Stretch (linear)



Rotate (linear)



Project (linear)

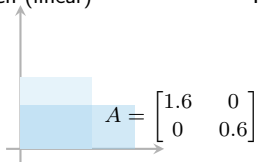


Key

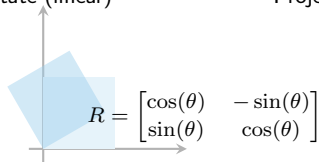
-  original vector
-  transformed/not-linear ex.
-  perpendicular drop

What Linear Models Can (and Cannot) Do

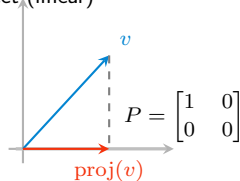
Stretch (linear)



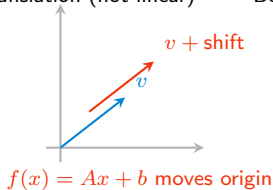
Rotate (linear)



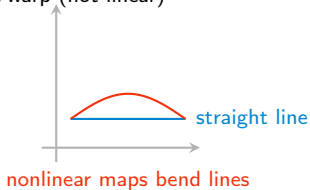
Project (linear)



Shift/translation (not linear)



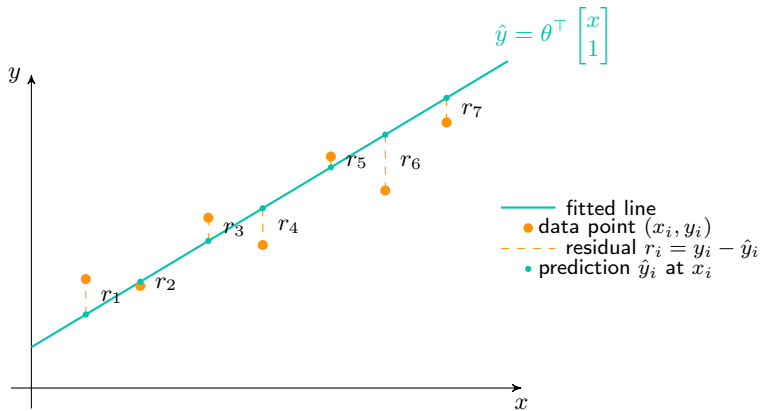
Bend/warp (not linear)



Key

- original vector
- transformed/not-linear ex.
- - perpendicular drop

Measuring Predictor Error in 2D (Linear Regression)

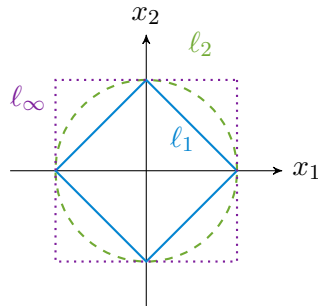


Loss Functions Measure Prediction Error

- The residual for data point i is $r_i = y_i - f_\theta(x_i)$.
- We measure total error via a *loss*:

$$\mathcal{L}(\theta) = \sum_{i=1}^n \|r_i\|_p$$

where $\|\cdot\|_p$ is the ℓ_p norm.



- ℓ_2 norm $\|x\|_2 = (\sum_i x_i^2)^{1/2}$: penalizes large errors strongly (smooth).
- ℓ_1 norm $\|x\|_1 = \sum_i |x_i|$: robust to outliers (sparse errors).
- ℓ_∞ norm $\|x\|_\infty = \max_i \{|x_i|\}$: minimizes worst-case error.

Linear Models: The Workhorse of ML

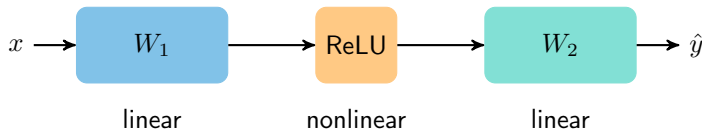
Linear Models Are Cool!



- Linear regression may look simple, but it's the foundation of modern ML.
- Every deep network layer, transformer block, and embedding lookup is built from linear maps:

$$h_{k+1} = W_k h_k + b_k \quad (\text{linear transformation}).$$

- Nonlinearities (ReLU, softmax, etc.) just compose these linear pieces.



Takeaway: Modern ML = many linear models stacked + nonlinearities.

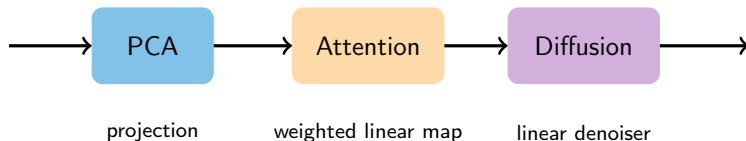
Linear Algebra is the Engine of Modern ML

- All large-scale ML systems are dominated by linear algebra:

matrix multiplications, projections, least-squares updates.

- Examples:

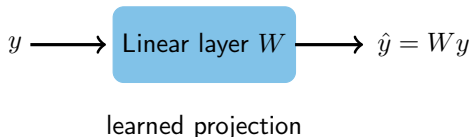
- ▶ Transformers: multi-head attention uses dot products $\rightarrow$ projections.
- ▶ PCA: dimensionality reduction = best low-rank linear approximation.
- ▶ Diffusion models: each denoising step fits a linear prediction to noisy data.



Message: Even billion-parameter models rely on the same math we use in least squares.

Linear Layers Are Learnable Projections

- In least squares, we *project* y onto the column space $\mathcal{C}(X)$ of X (given data).
- In neural networks, weights W define the subspace we project onto.
- Training = adjusting W so that projections align well with desired outputs.



Same idea: find the best projection, where the subspace (weights) changes with data.

Attention Heads Are Learnable Linear Projections

Key idea: Even in modern architectures like Transformers, the core operation inside each attention head is **linear in the inputs**.

Recall: what makes a model linear?

output is *weighted sum* of its inputs: $\hat{y} = \sum_i w_i x_i$.

Note: the weights can depend on context, but the combination itself is linear.

Attention does exactly this!

Each attention head: Output for item $i = \sum_j (\text{weight from } i \text{ to } j) \times (\text{embedded input } j)$.

- The model learns how to choose the weights (who pays attention to whom).
- Once the weights are set, the output is a **linear combination** of the inputs.

A Simple View: Linear Map on Learned Embeddings

Key idea: Once an attention head is trained, its weights are fixed. At inference, it performs a sequence of **linear transformations** on the input embeddings.

$$\text{output} = W f(X)$$

Interpretation:

- X : the input embedding (learned representation of the data)
- $f(X)$: a transformation that mixes or reweights embeddings based on context
- W : a fixed linear map learned during training



Big idea: Even in complex architectures like attention heads, the computation reduces to a *linear transformation* of the current embedding.

We addressed "why linear", so now why start with least squares?

Why Start with Least Squares?

- It's the simplest form of **empirical risk minimization**:
- It teaches:
 - ▶ Geometry of learning (projection, subspaces)
 - ▶ Optimization (gradients, convexity)
 - ▶ Regularization (bias–variance, ridge regression)
- These same ideas reappear in:
 - ▶ Linear classifiers (SVMs, logistic regression)
 - ▶ Neural networks (gradient descent on squared loss)
 - ▶ Reinforcement learning (least-squares temporal difference)

Once you understand least squares, you understand the logic behind all of these.

Empirical Risk Minimization (ERM): Quientessential ML Problem

Goal of supervised learning: Find a function that makes good predictions on new data.

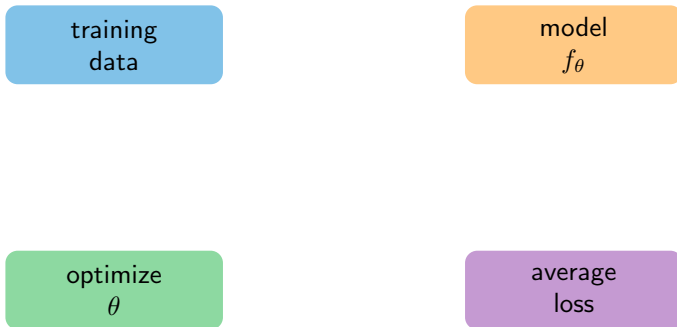
Challenge: We don't know the true data distribution; indeed, we only see a **finite** set of samples.

Idea of ERM:

Why it matters: ERM formalizes the idea of *fitting a model to data*.

- Almost every ML algorithm—from linear regression to deep networks—minimizes some form of empirical risk.
- Least squares is the simplest example.

ERM: The Big Picture



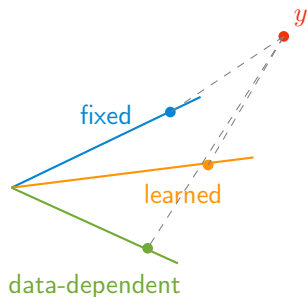
Takeaway: ERM formalizes model training as minimizing the average loss—the *empirical risk*—and the process repeats as new data arrive.

Big Picture: ML as Learning to Project

ML algorithms are understood as **projections onto subspaces**—some fixed, some learned. That is, they find the closest point in a subspace to the data.

Method	Projection target (subspace)	How it's chosen
Least Squares	Columns of X (given data)	Fixed by features
PCA	Top eigenvectors of $X^\top X$	Learned to maximize variance
Attention	Value vectors V weighted by similarity of q and K	Learned, data-dependent
Diffusion Models	Linear prediction of noise component	Learned through training

Big Picture: ML as Learning to Project



Color	Represents	Meaning in ML Context
■	Least Squares subspace	Fixed column space of X : the features you choose define the projection.
■	PCA subspace	Low-dimensional directions learned from data to maximize variance.
■	Attention subspace	Data-dependent subspace; each query q changes the projection space dynamically.

Core insight: All these methods find or use a subspace where predictions are “closest” to data; the difference is *how the subspace itself is learned*.

Onto the learning part: Least Squares

Least Squares: The Simplest Form of ERM & Supervised Learning

Least Squares Set-up

- Fitting a “linear regression model” is the simplest form of machine learning out there.
- Consider an $n \times d$ matrix $X \in \mathbb{R}^{n \times d}$ and vectors $y \in \mathbb{R}^n$, $\theta \in \mathbb{R}^d$
- **Goal:** find θ such that $X\theta = y$
- Interpretation:
 - ▶ training data $X \in \mathbb{R}^{n \times d}$; e.g., n is number of samples, d is number of ‘features’
 - ▶ $y \in \mathbb{R}^n$ contains ‘target values’, ‘labels’, or observations
 - ▶ $\theta \in \mathbb{R}^d$ is a set of feature weights

Example Application: Advertising Purchases

- Consider n demographic groups that we want to advertise to with a target # 'impressions' y for each group
- We purchase advertising in d different channels (e.g., different web publishers, radio, print, ...), in amounts given by $\theta \in \mathbb{R}^d$.
- Entry x_{ij} in matrix $X \in \mathbb{R}^{n \times d}$: # impressions in group i per dollar spent on advertising in channel j .
 - ▶ j th column of X gives the effectiveness or reach (in impressions per dollar) for channel j .
 - ▶ i th row of X shows which media demographic group i is exposed to.
- **Goal:** find θ such that $\|X\theta - y\|_2^2$ is as small as possible (minimized)

Overdetermined System of Equations $\rightarrow$ Least Squares Opt

- **Goal:** Find a solution to $X\theta = y$ —that is, find θ such that $X\theta = y$
- typically, X is a ‘tall’ matrix (so we get an ‘over-determined’ system); there are more equations (n) than variables to choose (d).
- there’s often no exact solution $\rightarrow$ formulate an **optimization problem** to find as *close* a solution as possible; i.e., an *least squares approximate solution*

Least Squares Formulation

Least Squares Formulation: Empirical Risk Minimization

- $X \in \mathbb{R}^{n \times d}$: the *design matrix*, rows are data points $x_i^\top$.
- $y \in \mathbb{R}^n$: vector of targets.
- ℓ_2 loss is smooth, convex, and differentiable $\rightarrow$ analytic solution exists.

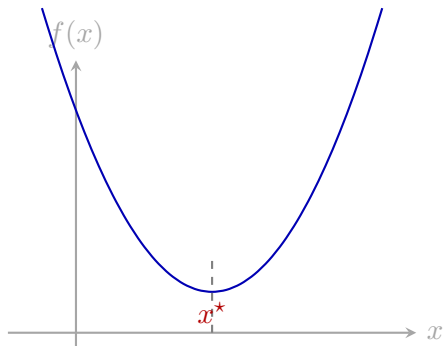
Finding Minima and Convexity

Goal: Find θ^* that minimizes a differentiable function $f : \mathbb{R}^d \rightarrow \mathbb{R}$.

Conditions for Minima

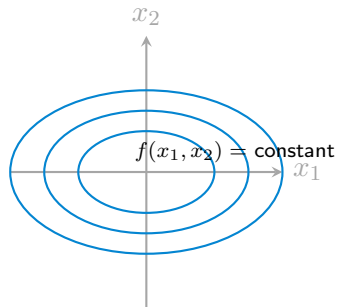
Finding Minima in One Dimension

For a smooth function $f : \mathbb{R} \rightarrow \mathbb{R}$, a minimum occurs when:



Minima in Two Dimensions

Now $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ has a gradient and Hessian:



General Convexity Condition via Eigenvalues

For any symmetric Hessian $H \in \mathbb{R}^{d \times d}$,

$$H \text{ is positive definite} \iff v^\top H v > 0 \text{ for all } v \neq 0.$$

Checking this directly is hard, so we use **eigenvalues**.

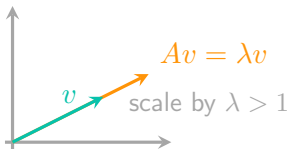
Eigenvalues and Eigenvectors

For a square matrix $A \in \mathbb{R}^{d \times d}$:

Geometric meaning:

- A scales (stretches or flips) v without changing its direction.
- The eigenvalues describe how much A stretches space along special directions.
- If $\lambda = 0$, that direction is *flattened* $\rightarrow$ indicates "rank deficiency".

Rank and eigenvalues:



Why Eigenvalues Indicate Positive Definiteness

Let $A \in \mathbb{R}^{d \times d}$ and consider the quadratic form:

Interpretation:

- Each eigenvector v defines a special direction in space.
- Along that direction, A scales by λ and stretches by λ^2 in $A^\top A$.
- If $\lambda^2 > 0$ for all eigenvalues, then $A^\top A$ is **positive definite** (i.e. $A^\top A \succ 0$).

Therefore:

What are we missing?

- First, we have $v^\top H v$, but we just looked at a condition on matrices like $A^\top A$. So can we write H as $A^\top A$ for some A ?
 - ▶ Yes! If H is **symmetric** and **positive semidefinite** (i.e. $H \succeq 0 \iff x^\top H x \geq 0, \forall x$).
 - ▶ We know its symmetric because of its structure as a Hessian ($H = H^\top$)
- What is **linear independence** of eigenvector?

Independence of Eigenvectors and Basis Formation

Linear Independence

Independence of Eigenvectors and Basis Formation

Theorem

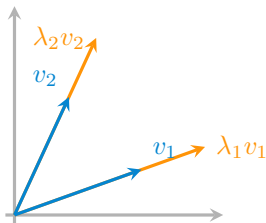
If $Av_i = \lambda_i v_i$ and $Av_j = \lambda_j v_j$ with $\lambda_i \neq \lambda_j$, then v_i and v_j are linearly independent.

Proof sketch:

Independence of Eigenvectors and Basis Formation

Facts

- For $A \in \mathbb{R}^{d \times d}$ with d distinct eigenvalues, its eigenvectors $v_1, \dots, v_d$ are linearly independent.
- If A has d distinct eigenvalues, its eigenvectors form a **basis** for $\mathbb{R}^d$.
- Distinct eigenvalues guarantee independent eigenvectors, which form a **basis** that **diagonalizes** A .

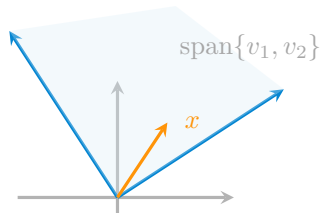


What is a Basis?

Definition of Basis

Key Ideas:

- “Span” $\implies$ the basis vectors *reach* all of V .
- “Linear independence” $\implies$ no vector in the set is redundant.
- The number of basis vectors = **dimension** of V .



Independence of Eigenvectors and Basis Formation

Why this matters:

Interpretation:

- Eigenvectors give A 's *natural coordinate system*.
- Their independence guarantees we can describe any vector as

$$x = \alpha_1 v_1 + \cdots + \alpha_n v_d.$$

- A then acts by simply scaling each coordinate by λ_i .

Where are we

- We have defined linear independence, and even shown how to diagonalize a matrix if it has independent eigenvectors.
- Now we want to come back to this Hessian matrix and questions about curvature.
- To apply the eigenvalue check, we need to show that Hessians can be decomposed as $H = A^\top A$ for some A .

Why Symmetric Matrices Can Be Diagonalized

Claim (Spectral Theorem)

Intuition:

- Symmetry $\Rightarrow$ all eigenvalues are **real**.
- Symmetry $\Rightarrow$ eigenvectors are **orthogonal** (i.e., $\langle v_i, v_j \rangle = 0$ for $i \neq j$).
- Then $Q^\top Q = I$ so that $HQ = Q\Lambda \implies H = Q\Lambda Q^\top$.

Checking Convexity with the Hessian

$$H(\theta) = \nabla^2 f(\theta) = \begin{bmatrix} \frac{\partial^2 f}{\partial \theta_1^2} & \cdots & \frac{\partial^2 f}{\partial \theta_1 \partial \theta_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial \theta_d \partial \theta_1} & \cdots & \frac{\partial^2 f}{\partial \theta_d^2} \end{bmatrix} \quad (\text{Hessian})$$

$H(\theta) \succ 0 \iff v^\top H(\theta) v > 0 \quad \forall v \neq 0 \implies f$ is **strictly convex** at that point

Eigenvalue test:

Eigenvalues and Curvature: PD, PSD, and Indefinite

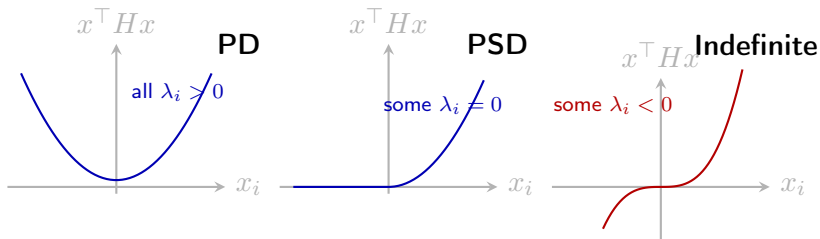
For a symmetric matrix H , curvature along direction v is measured by

$$v^\top H v = \lambda \|v\|_2^2.$$

$\lambda_i > 0 \Rightarrow$ curves upward (bowl)

$\lambda_i = 0 \Rightarrow$ flat direction (ridge)

$\lambda_i < 0 \Rightarrow$ curves downward (saddle)



Takeaway:

Summary: Positive Definiteness, Convexity, and Optimality

- Optimality: $\nabla f(\theta^*) = 0$ and $H(\theta^*) \succ 0 \implies \theta^*$ is a local minimum.
- Convexity: $H(\theta) \succ 0$ for all $\theta \implies f$ is convex $\implies$ any stationary point is a global minimum.
- Positive definiteness: $H \succ 0 \iff$ all eigenvalues $\lambda_i > 0$.
 - ▶ 1D: $f''(x) > 0$.
 - ▶ 2D: both diagonal entries > 0 .
 - ▶ General d : all eigenvalues of H positive $\implies$ convex function.

Solving Least Squares (Coordinate Form)

For one data point: $f_{\theta}(x_i) = \theta_1 x_{i1} + \theta_2 x_{i2} + \cdots + \theta_d x_{id} = \theta^{\top} x$.

$$\text{minimize } \mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \theta_1 x_{i1} - \theta_2 x_{i2} - \cdots - \theta_d x_{id})^2.$$

Take partial derivatives with respect to each θ_j :

Solving Least Squares (Vector Form)

$$\mathcal{L}(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 = \frac{1}{n} (y - X\theta)^\top (y - X\theta).$$

Coordinate vs. Vector Gradient (2D Example)

Consider two features: $x_i = [x_{i1}, x_{i2}]^\top$ and parameters $\theta = [\theta_1, \theta_2]^\top$.

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - x_{i1}\theta_1 - x_{i2}\theta_2)^2 = \frac{1}{n} \|y - X\theta\|_2^2.$$

Coordinate form:

$$\frac{\partial \mathcal{L}}{\partial \theta_1} = -\frac{2}{n} \sum_{i=1}^n x_{i1}(y_i - x_{i1}\theta_1 - x_{i2}\theta_2),$$

$$\frac{\partial \mathcal{L}}{\partial \theta_2} = -\frac{2}{n} \sum_{i=1}^n x_{i2}(y_i - x_{i1}\theta_1 - x_{i2}\theta_2).$$

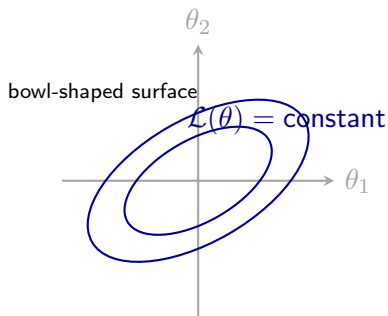
Vector form:

$$\begin{aligned} \nabla_{\theta} \mathcal{L}(\theta) &= -\frac{2}{n} X^\top (y - X\theta) \\ &= -\frac{2}{n} \begin{bmatrix} \sum_i x_{i1}(y_i - x_{i1}\theta_1 - x_{i2}\theta_2) \\ \sum_i x_{i2}(y_i - x_{i1}\theta_1 - x_{i2}\theta_2) \end{bmatrix}. \end{aligned}$$

Uniqueness: Convexity of the Least Squares Objective

$$\min_{\theta} \mathcal{L}(\theta) = \frac{1}{n} \|y - X\theta\|_2^2 = \frac{1}{n} (y - X\theta)^\top (y - X\theta) \implies \theta^\star = (X^\top X)^{-1} X^\top y.$$

Convexity Intuition (2D Example)



Takeaway: Least squares is convex because its Hessian $\frac{2}{n}X^\top X$ is positive definite when X has full column rank.

What happens if some directions have zero curvature?

Next time

Today we covered:

- Convexity, Hessians, and curvature
- Linear independence, basis, and dimension
- Why symmetric matrices can be diagonalized
- Checking convexity with the eigenvalue test
- Solving least squares by setting the gradient to zero

Next time we will cover:

- Linear Algebra Concepts: Matrix Rank and the Pseudoinverse, Under- vs Over-determined systems
- Projections onto subspaces and connections to least squares
- Meaning/interpretation of least squares as a ML model